University of Fort Hare
Department of Computer Science

CSC313: Object-Oriented Programming

Assignment 2

Implementing OOP Concepts, Exception Handling, Interfaces & Abstract Classes

Submitted By	**Group 4**
Submission Date	**10 May 2026**
Lecturer	**Dr Nomnga**
GitHub Repository	**https://github.com/Sisamke/Assignment2.git**

1. Group Details

#	Full Name	Student Number	Role	Contribution
1	Sinesipho Malgas	202322940	Group Leader	Development / Documentation
2	Phumelela Jojile	224005425	Member	Development / Documentation
3	Sisamkele Mqushwane	202393171	Member	Development / Documentation
4	Lilitha Dlaku	202353752	Member	Development / Documentation
5	Nokuthula Kukhanya Mpofu	223051614	Member	Development / Documentation
6	Natasha Lugalo	224045823	Member	Development / Documentation

2. Assignment Overview

This assignment requires the development of an Android application using Android Studio to facilitate the buying and selling of second-hand textbooks. The application demonstrates core Object-Oriented Programming (OOP) principles including Interfaces, Abstract Classes, Exception Handling, Encapsulation, Inheritance, and Polymorphism, as specified in the CSC313 course requirements.

The application targets the student community at the University of Fort Hare, providing a platform where sellers can list their textbooks and buyers can browse and search listings efficiently.

2.1 Functional Requirements

The Android application implements the following core features:

- Browse all available textbooks currently listed for sale
- List a textbook for sale by providing title, author, module code, number of copies, selling price, and banking information
- Submit and confirm listings, with duplicate entry prevention using Exception Handling
- Search for textbooks by seller name or book title

2.2 OOP Concepts Applied

- Interfaces (Listable) — defining contracts for listing behaviour
- Abstract Classes (Product) — shared fields and methods across book types
- Inheritance — Textbook extends Product
- Encapsulation — private fields with public getters and setters

- Polymorphism — dynamic dispatch through interface and abstract class references
- Exception Handling — custom DuplicateTextbookException and InvalidInputException

3. System Design

3.1 Architecture Overview

The application follows a layered architectural pattern separating the UI layer, business logic, and data management. This ensures maintainability, testability, and clear separation of concerns.

- Presentation Layer — Android Activities and Fragments handling user interactions and UI rendering
- Business Logic Layer — Java classes implementing OOP concepts (interfaces, abstract classes, exception handling)
- Data Layer — In-memory data storage using ArrayList and HashMap for textbook listing management

3.2 Class Design

3.2.1 Interface: Listable

Defines the contract for any object that can be listed on the marketplace:

- listItem() — registers a new item in the data store
- getDetails() — returns a formatted string representation of the item
- validateListing() — validates all fields before submission

3.2.2 Abstract Class: Product

Serves as the base class for all sellable items. Contains common fields:

- title (String) — the name of the product
- author (String) — the author or creator
- price (double) — the selling price
- copies (int) — number of copies available
- sellerName (String) — the name of the listing seller

3.2.3 Concrete Class: Textbook

Extends Product and implements Listable, adding textbook-specific attributes:

- moduleCode (String) — the module/course code the book is used for
- bankingDetails (String) — seller payment information
- Overrides equals() and hashCode() to enable duplicate detection

3.3 Screen Design

The application consists of the following screens/activities:

Screen	Description	Key Components
Home Screen	Landing page with navigation options for buyers and sellers	NavigationDrawer / BottomNav
Browse Screen	Displays all available textbook listings in a scrollable list	RecyclerView, TextbookAdapter
List a Book Screen	Form for sellers to enter book details and banking information	EditText fields, Submit Button
Search Screen	Search bar with filtering by seller name and/or book title	SearchView, filtered RecyclerView
Confirmation Dialog	Modal shown on successful listing submission	AlertDialog

4. Methodology

4.1 Development Approach

The group adopted an Agile-inspired development approach, dividing work into small iterative cycles with regular check-ins. Tasks were distributed among members based on individual strengths and agreed upon during the initial group meeting held on 18 April 2026.

4.2 Task Distribution

Member	Responsibility	Status
Sinesipho Malgas	Project coordination, class hierarchy design, GitHub management	Completed
Phumelela Jojile	Browse screen implementation and RecyclerView adapter	Completed
Sisamkele Mqushwane	List a Book screen, form validation and submission logic	Completed
Lilitha Dlaku	Exception handling classes and duplicate detection logic	Completed
Nokuthula Kukhanya Mpofu	Search screen and filtering implementation	Completed
Natasha Lugalo	UI/UX design, Home Screen, and project documentation	Completed

4.3 Version Control Strategy

All code was hosted on a dedicated GitHub repository created for Assignment 2. The following practices were observed:

- Each member committed their own work directly from their local machine
- Descriptive commit messages were used to track progress
- The lecturer (Dr Nomnga) was added as a collaborator on the repository
- A README.md file was included with project setup instructions

5. Implementation

5.1 OOP Structure Implementation

5.1.1 Interface and Abstract Class

The Listable interface was defined with three method signatures: listItem(), getDetails(), and validateListing(). The abstract class Product holds the common fields (title, author, price, copies, sellerName) with private visibility, exposing them through standard Java getters and setters to enforce Encapsulation.

The concrete Textbook class extends Product and implements Listable, providing concrete implementations of all abstract and interface methods. The equals() method compares title and author fields to identify duplicates, while hashCode() ensures consistent HashMap behaviour.

5.1.2 Exception Handling

Two custom exception classes were implemented:

- DuplicateTextbookException — thrown when a textbook with the same title and author already exists in the data store. The message informs the user that the listing already exists.
- InvalidInputException — thrown when form fields are empty, price is negative or zero, or the number of copies is less than one.

All data submission and retrieval operations are wrapped in try-catch blocks within the relevant Activity classes. Error messages are displayed to the user using Android Toast messages or AlertDialogs.

5.2 Data Management

An ArrayList<Textbook> serves as the primary in-memory data store for all textbook listings. A HashSet<String> of composite keys (title + author) is maintained separately to facilitate O(1) duplicate detection before any new listing is added.

5.3 Search Implementation

The search feature filters the in-memory ArrayList using Java stream operations. The filter checks whether the search query (case-insensitive) matches either the seller name or the book title. The filtered results are passed to the RecyclerView adapter, which refreshes the displayed list dynamically.

5.4 GitHub Repository

Repository URL: https://github.com/Sisamke/Assignment2.git

The repository contains the complete Android Studio project, including all source files, layout XML files, and the README.md setup guide. Regular commits from all six members are reflected in the commit history.

6. Testing

6.1 Testing Approach

Testing was conducted manually using the Android Emulator (Pixel 6 API 33) within Android Studio. Each functional requirement was verified through a series of test scenarios, and edge cases were tested specifically around exception handling and duplicate detection.

6.2 Test Cases

#	Test Case	Input	Expected Result	Outcome
1	Browse all listings	Navigate to Browse Screen	All listed textbooks displayed in RecyclerView	Pass
2	List a new textbook	Valid title, author, price, copies, banking details	Confirmation dialog shown; book added to list	Pass
3	Duplicate textbook listing	Same title and author as an existing listing	DuplicateTextbookException caught; error shown to user	Pass
4	Invalid input — empty fields	Submit form with one or more blank fields	InvalidInputException caught; user prompted to fix fields	Pass
5	Invalid input — negative price	Price entered as -50	InvalidInputException caught; error message displayed	Pass
6	Search by book title	Enter partial book title in search bar	Matching textbooks shown; non-matches hidden	Pass
7	Search by seller name	Enter partial seller name	All books from matching sellers displayed	Pass
8	Search with no matches	Enter a name with no matching records	Empty list shown with appropriate message	Pass

6.3 Testing Summary

All 8 test cases passed successfully during manual testing on the Android Emulator. The exception handling logic correctly prevented invalid data from being stored, and the search feature returned accurate results for both full and partial query strings. No critical bugs were identified at the time of submission.

7. Challenges Encountered

7.1 Understanding Interfaces vs Abstract Classes

One of the initial challenges was determining the correct use of interfaces versus abstract classes in the context of the application. The group spent time in the first meeting aligning on the design — specifically deciding that Listable should be an interface (a contract) while Product should be an abstract class (shared implementation). Consulting the course material and online Java OOP documentation helped the group reach consensus.

7.2 Duplicate Detection Logic

Implementing reliable duplicate detection required careful overriding of the equals() and hashCode() methods in the Textbook class. Initially the duplicate check was not working correctly because the default Object.equals() was being called rather than the overridden version. This was resolved by ensuring the equals() method explicitly compared the title and author fields (case-insensitively) rather than object references.

7.3 Exception Handling in Android Activities

Integrating custom exception handling within Android Activity lifecycle methods required understanding where to place try-catch blocks. Some members were unfamiliar with the nuances of Android's event-driven model, causing confusion around where exceptions should be caught versus where they should propagate. Regular code reviews within the group resolved inconsistencies in exception handling patterns.

7.4 RecyclerView Adapter and Live Data Updates

Refreshing the RecyclerView when a new book was listed or when search results changed proved more difficult than expected. Initially, the list was not updating visually after a new listing was submitted. The issue was traced to not calling notifyDataSetChanged() on the adapter after modifying the underlying data source. This was corrected and the adapter now reflects live changes accurately.

7.5 Collaboration and Merge Conflicts on GitHub

Working collaboratively on the same codebase via GitHub introduced occasional merge conflicts, particularly in the MainActivity.java and activity_main.xml files. The group established a practice of pulling the latest changes before starting work and communicating through the group chat before modifying shared files. These practices reduced conflicts significantly for the latter half of development.

7.6 Time Management

Balancing this assignment alongside other academic commitments was a challenge for several group members. The deadline of 12 May 2026 required early planning and task delegation to ensure the project was completed on time. The group met three times in total (in-person and virtually) to synchronise progress and resolve blockers.